

Comprehensive Summary of All Lectures

Course: Artificial Intelligence and Expert Systems (CCE3219)

Dr. Hany El-Ghaish & Dr. Reda Elbasiony

PART 1: ARTIFICIAL INTELLIGENCE FUNDAMENTALS

Lecture 1 — Introduction to Artificial Intelligence

What is AI?

Artificial Intelligence is approached from four different perspectives:

1. **Thinking Humanly** — Cognitive science: understanding the brain as an information-processing machine. Methods include introspection, behavioral testing, and brain imaging (cognitive neuroscience).
2. **Acting Humanly — The Turing Test** (Turing, 1950): A computer passes this test if it can fool 30% of human judges in a 5-minute conversation. Required capabilities include:
 - Natural Language Processing
 - Knowledge Representation
 - Automated Reasoning
 - Machine Learning

Criticisms of the Turing Test: Some human behavior is not intelligent; some intelligent behavior is not human; the "Chinese Room" argument (simulating intelligence ≠ having intelligence).

3. **Thinking Rationally** — Using logic and formal notation to describe and solve problems (the "Logician" approach). Problems: computational complexity and difficulty formalizing real-world knowledge.
4. **Acting Rationally — Rational Agents:** A rational agent maximizes its expected utility (best expected outcome). This is the most practical and dominant approach in modern AI.

Real-World AI Examples

- **IBM Watson** — won Jeopardy! (2011)
- **Google Self-Driving Cars**
- **Natural Language Processing** — speech recognition, machine translation
- **Computer Vision** — OCR, face detection, visual search

- **Games** — IBM Deep Blue defeated Kasparov in chess (1997); checkers was "solved" in 2007
 - **Logistics & Planning** — US Gulf War scheduling (1991), NASA Mars rover planning
 - **Information Agents** — search engines, spam filters, fraud detection
 - **Robotics** — Mars rovers, autonomous vehicles, humanoid robots
-

Lecture 2 — Rational Agents & Introduction to Search

Agents

An **agent** perceives its environment through **sensors** and acts through **actuators**.

- **Agent function:** maps percept history → actions
- **Agent = Architecture + Program**

PEAS Framework (Task Environment Specification)

Component	Description
Performance Measure	Objective criterion for success
Environment	The world the agent operates in
Actuators	How the agent acts
Sensors	How the agent perceives

Example (Automated Taxi): Performance = safe, fast, legal trip; Environment = roads, traffic; Actuators = steering, brakes; Sensors = cameras, GPS, sonar.

Environment Types

- **Fully Observable vs. Partially Observable**
- **Deterministic vs. Stochastic**
- **Episodic vs. Sequential**
- **Static vs. Dynamic** (Semidynamic: environment doesn't change, but score does)
- **Discrete vs. Continuous**
- **Single Agent vs. Multi-Agent**
- **Known vs. Unknown**

Hierarchy of Agent Types

1. **Simple Reflex Agent** — Acts only on current percept, ignores history
2. **Model-Based Reflex Agent** — Maintains internal state of unobservable aspects

3. **Goal-Based Agent** — Selects actions based on goal information
4. **Utility-Based Agent** — Uses a utility function to evaluate desirability of states

Search Problem Components

- **Initial State:** Where the agent starts
- **Actions:** What the agent can do
- **Transition Model:** Result of performing an action
- **Goal State:** What the agent is trying to achieve
- **Path Cost:** Cost of reaching the goal (sum of non-negative step costs)
- **Optimal Solution:** Sequence of actions with the lowest path cost

Tree Search Algorithm

1. Initialize fringe with the starting state
2. While fringe is not empty:
 - Choose a node to expand
 - If it's the goal → return solution
 - Else expand and add children to the fringe

Search Strategy Evaluation Criteria

- **Completeness:** Always finds a solution if one exists?
 - **Optimality:** Always finds the least-cost solution?
 - **Time Complexity:** Number of nodes generated ($O(b^d)$)
 - **Space Complexity:** Maximum nodes in memory
-

Lecture 3 — Uninformed (Blind) Search Strategies

These strategies use only information from the problem definition — no extra knowledge about how close a state is to the goal.

1. Breadth-First Search (BFS)

- Expands the **shallowest** unexpanded node
- Uses a **FIFO queue**
- **Complete:** Yes (if branching factor b is finite)
- **Optimal:** Yes (if cost = 1 per step)
- **Time:** $O(b^{(d+1)})$
- **Space:** $O(b^{(d+1)})$ — keeps all nodes in memory (biggest problem)

2. Uniform-Cost Search (UCS)

- Expands the **lowest-cost** unexpanded node
- Uses a **priority queue** ordered by path cost
- **Complete:** Yes (if step cost > 0)
- **Optimal:** Yes — expands nodes in increasing order of path cost
- **Time & Space:** $O(b^{(C^*/\epsilon)})$ where C^* is optimal cost, ϵ is minimum step cost
- Equivalent to BFS when all step costs are equal

3. Depth-First Search (DFS)

- Expands the **deepest** unexpanded node
- Uses a **LIFO stack**
- **Complete:** No (fails in infinite-depth or looping spaces)
- **Optimal:** No
- **Time:** $O(b^m)$ — terrible if max depth $m \gg$ goal depth d
- **Space:** $O(bm)$ — **linear space** (major advantage)

4. Depth-Limited Search

- DFS with a maximum depth limit L
- Avoids infinite paths but adds incompleteness if $L < d$

5. Iterative Deepening Search (IDS)

- Combines DFS space efficiency with BFS completeness
- Runs DFS repeatedly with increasing depth limits $(0, 1, 2, \dots)$
- **Complete:** Yes
- **Optimal:** Yes (uniform step costs)
- **Time:** $O(b^d)$ — same as BFS asymptotically
- **Space:** $O(bd)$ — same as DFS (linear)

6. Informed Search: Greedy Best-First

- Expands the node **closest to the goal** (by heuristic estimate $h(n)$)
- Fast but not optimal (can get trapped in dead ends)

7. A* Search

- Combines Uniform-Cost (backward cost $g(n)$) and Greedy (forward cost $h(n)$)
- **$f(n) = g(n) + h(n)$**
- **Complete & Optimal** if the heuristic h is **admissible** (never overestimates the true cost)
- Expands mainly toward the goal, hedging bets to ensure optimality

- Applications: video games, routing/pathing, robot motion planning, machine translation

Admissible Heuristic: $h(n) \leq h^*(n)$ (true cost to goal). Inadmissible (pessimistic) heuristics break optimality.

Lecture 4 (5) — Constraint Satisfaction Problems (CSPs)

What is a CSP?

- A special type of search problem where the **goal itself** is what matters (not the path)
- **State:** defined by **variables X_i** with values from a **domain D**
- **Goal test:** a set of **constraints** specifying allowable value combinations
- A **consistent assignment** violates no constraints
- A **complete assignment** assigns values to all variables
- A **solution** is a consistent, complete assignment

Types of Constraints

- **Unary:** involve a single variable (equivalent to reducing the domain)
- **Binary:** involve pairs of variables (e.g., neighboring regions must differ in color)
- **Higher-order:** involve 3 or more variables
- **Soft constraints (Preferences):** have a cost — lead to constrained optimization

CSP Examples

- **Map Coloring:** color regions so no two adjacent regions share a color
- **N-Queens:** place N queens on an $N \times N$ board so no two attack each other

Real-World CSPs

- Timetabling, scheduling, hardware configuration, circuit layout, transportation

Solving CSPs

Backtracking Search = DFS + variable-ordering + fail-on-violation

- Assign one variable at a time (assignments are commutative)
- Check constraints as you go (incremental goal test)
- Backtrack when a violation is found
- Can solve N-queens for $N \leq 25$

Improving Backtracking:

- **Filtering (Forward Checking):** After assigning a variable, cross off values in neighboring variables' domains that would violate a constraint. Detect failure early.
 - **Ordering:** Choose which variable to assign next (Most Constrained Variable); choose which value to try first (Least Constraining Value)
 - **Structure Exploitation:** Use problem structure to decompose into independent subproblems
-

PART 2: MACHINE LEARNING

Lecture 7 — Introduction to Machine Learning

What is Machine Learning?

"Learning is any process by which a system improves performance from experience." — Herbert Simon

Tom Mitchell's Definition (1998): ML is the study of algorithms that:

- Improve their **performance P**
- at some **task T**
- with **experience E**

A well-defined learning task = $\langle P, T, E \rangle$

Traditional Programming: Data + Program $\rightarrow$ Computer $\rightarrow$ Output

Machine Learning: Data + Output $\rightarrow$ Computer $\rightarrow$ Program (learns the program)

When to Use Machine Learning?

- Human expertise does not exist (navigating Mars)
- Humans cannot explain their expertise (speech recognition)
- Models must be customized (personalized medicine)
- Models are based on huge amounts of data (genomics)

Types of Learning

Type	Description	Examples
Supervised	Training data with labels	Classification, Regression
Unsupervised	Training data without labels	Clustering, ICA
Semi-supervised	Training data + a few labels	—
Reinforcement	Rewards from sequence of actions	Game playing, robotics

Supervised Learning

- **Regression:** Predict a continuous value (e.g., house price)
- **Classification:** Predict a category (e.g., cancer: malignant/benign)
- Input x can be multi-dimensional (each dimension = a feature/attribute)

Unsupervised Learning

- Find hidden structure in unlabeled data
- Examples: clustering genes by similarity, market segmentation, social network analysis
- **Independent Component Analysis (ICA):** separate combined signals into original sources

Reinforcement Learning

- Learn a **policy** (mapping from states $\rightarrow$ actions)
- Agent receives delayed rewards from sequences of actions
- Agent-environment interface: at each step t , the agent observes state s_t , takes action a_t , and receives reward r_{t+1}

ML System Design

1. Choose training experience
2. Choose target function
3. Choose function representation
4. Choose learning algorithm

Three Components of Every ML Algorithm

- **Representation:** How to represent the learned function (linear models, decision trees, neural networks, Bayesian networks, etc.)
- **Optimization:** How to search for the best parameters (gradient descent, dynamic programming, evolutionary computation)

- **Evaluation:** How to measure performance (accuracy, precision/recall, squared error, entropy, etc.)
-

Lecture 8 — Supervised Learning: Classification & Regression

Classification

- Predict a **categorical** output label from features
- Common algorithms: Decision Trees, Naïve Bayes, K-Nearest Neighbors, SVM, Neural Networks

Linear Regression

- Predict a **continuous** output y from input features x
- Model: $\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d$ (weighted sum of features + bias)
- In matrix form: $\hat{y} = Xw$

Objective: Minimize the **Residual Sum of Squares (RSS)**:

$$RSS(w) = (y - Xw)^T(y - Xw)$$

Two ways to solve:

1. **Closed-Form (Normal Equations):**
 $X^T X \hat{w} = X^T y \rightarrow \hat{w} = (X^T X)^{-1} X^T y$
 2. **Gradient Descent** — iteratively move in the direction of the negative gradient:
 - $w(t+1) \leftarrow w(t) - \eta \cdot \nabla RSS(w(t))$
 - Update per feature: $w_j(t+1) \leftarrow w_j(t) + 2\eta \cdot \sum x_i[j](y_i - \hat{y}_i)$
 - Repeat until convergence (gradient norm $< \epsilon$)
-

Lecture 11 — Classification Performance Measures

Confusion Matrix

A table summarizing model predictions vs. actual labels (for binary classification):

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

Key Metrics

- **Accuracy** = $(TP + TN) / \text{Total}$
Limitation: Unreliable for imbalanced datasets
- **Precision** = $TP / (TP + FP)$
Of all predicted positives, what fraction were actually positive?
- **Recall (Sensitivity)** = $TP / (TP + FN)$
Of all actual positives, what fraction did the model detect?
- **F1-Score** = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
Harmonic mean of precision and recall; better than arithmetic mean for imbalanced cases
- **Specificity** = $TN / (TN + FP)$
Opposite of Recall; proportion of actual negatives correctly identified
- **AUC-ROC** (Area Under Curve - Receiver Operating Characteristic):
Plots TPR (Recall) vs. FPR (1 - Specificity) at various thresholds; higher AUC = better model
- **Log-Loss** (Cross-Entropy Loss): Measures uncertainty of predictions; accounts for predicted probability values, not just class labels

Regression Performance Metrics

- **Mean Absolute Error (MAE)**: Average absolute difference between actual and predicted
 - **Mean Squared Error (MSE)**: Average squared difference (penalizes large errors more)
 - **R² (R-Squared)**: Goodness of fit; proportion of variance explained by the model
-

Clustering (Unsupervised Learning) — K-Means

Types of Clustering Algorithms

Type	Approach	Strengths	Weaknesses
Partitional (K-Means, K-Medoids)	Divide into k non-overlapping groups	Scalable, works for spherical clusters	Poor for complex shapes/different densities
Hierarchical (Agglomerative/Divisive)	Bottom-up or top-down merging	Interpretable dendrogram, fine-grained detail	Computationally expensive, sensitive to noise
Distribution-Based (Gaussian Mixtures / EM)	Statistical model fitting	Captures correlation/dependence	Risk of overfitting, assumes specific distributions
Density-Based (DBSCAN, OPTICS)	Identify dense regions	Non-spherical clusters, outlier-resistant	Poor for high-dimensional or varying-density data

K-Means Algorithm

1. Choose k (number of clusters) and initialize k centroids randomly
 2. **Assign** each point to the nearest centroid
 3. **Update** each centroid to the mean of its assigned points
 4. Repeat 2-3 until convergence
- Non-deterministic (results vary with initialization)
 - Works best for spherical, equally-sized clusters

PART 3: COMPUTER VISION

CV Lectures — Image Filtering

Point Operations vs. Filters

- **Point operations:** Each output pixel depends only on the same input pixel (brightness/contrast adjustment, thresholding)
- **Filters:** Each output pixel depends on a **neighborhood** of input pixels

Uses of Filtering

- Image enhancement (denoising, sharpening)

- Feature extraction (edges, textures)
- Pattern detection (template matching)

Common Types of Noise

- **Salt and Pepper Noise:** Random black and white pixels
- **Impulse Noise:** Random white pixels only
- **Gaussian Noise:** Pixel values vary according to a normal distribution

Linear Filtering (Correlation / Convolution)

- Replace each pixel with a **weighted combination** of its neighbors
- Specified by a **filter kernel** (matrix of weights)
- **Cross-correlation:** Slide the kernel directly over the image
- **Convolution:** Flip the kernel in both dimensions, then cross-correlate
- Properties: Commutative, Associative, Distributive over addition, Linear scalability

Box Filter

- Averages all pixels in the neighborhood with equal weights
- Simple but non-isotropic; causes ringing due to sharp frequency cutoff

Gaussian Filter

- Weighted average giving higher importance to nearby pixels
- Controlled by σ (standard deviation): larger σ = more smoothing
- Better low-pass filter than box filter; isotropic

Properties of Smoothing Filters

- Non-negative coefficients
- Sum to 1 (preserve image range)
- Remove high-frequency components ("low-pass filter")

Difference Filters

- Have some negative coefficients
- Enhance local intensity changes (edges, sharpening)
- Applications: edge detection, image sharpening

Non-Linear Filters

Median Filter:

- Replaces each pixel with the **median** of its neighborhood

- Does NOT introduce new pixel values
- Excellent at removing **salt-and-pepper noise**
- **Edge-preserving** — better than Gaussian for impulse noise

Weighted Median Filter: Replicate pixel values proportional to integer weights before taking the median

Boundary Handling Options (OpenCV)

- `BORDER_CONSTANT`: pad with a fixed value
 - `BORDER_REPLICATE`: replicate edge pixels
 - `BORDER_REFLECT`: reflect the image at the border
 - `BORDER_WRAP`: wrap around
-

CV Lectures — Edge Detection

What Causes Edges?

Edges arise from: surface depth discontinuities, surface orientation changes, reflectance changes (color/texture), illumination changes (shadows).

Fundamental Concept

An edge is a place of **rapid change in image intensity**. Detected using image gradients (derivatives).

Image Gradient = vector of partial derivatives ($\partial I / \partial x, \partial I / \partial y$)

- **Gradient magnitude** = $\sqrt{(\partial I / \partial x)^2 + (\partial I / \partial y)^2}$
- **Gradient orientation** = $\arctan(\partial I / \partial y / \partial I / \partial x)$

Common Edge Detection Filters

- **Prewitt:** Equal weights, gradient + smoothing
- **Sobel:** Center row/column has higher weight; better noise suppression
- **Roberts:** Computes gradients along diagonal directions

Effect of Noise

Noise corrupts the derivative signal. Solution: **smooth first, then differentiate**.

Key insight: **Derivative of Gaussian (DoG)** = smooth and differentiate in one step.

The scale parameter σ controls edge detection scale:

- Larger $\sigma \rightarrow$ detects larger-scale edges
- Smaller $\sigma \rightarrow$ detects finer features

Canny Edge Detection (Multi-Stage)

1. **Filter** the image with derivative of a Gaussian
 2. **Compute** gradient magnitude and orientation
 3. **Non-Maximum Suppression**: Thin wide edge ridges to single-pixel width by suppressing non-local-maxima along the gradient direction
 4. **Hysteresis Thresholding** (double threshold):
 - Use a **high threshold** to start edge curves
 - Use a **low threshold** to continue them
 - This links strong and weak edges intelligently
-

CV Lectures — Local Features

Purpose of Local Features

Find points in an image that can be:

- Reliably located across different images
- Precisely localized
- Robustly matched

Applications: Image alignment, 3D reconstruction, motion tracking, object recognition

Three Main Components

1. **Detection**: Identify interest points (keypoints)
2. **Description**: Construct a feature vector (descriptor) for the region around each point
3. **Matching**: Find correspondences between descriptors in different images

Characteristics of a Good Feature

- **Repeatability**: Same feature found under geometric/photometric changes
- **Saliency**: Distinctive and uniquely describable
- **Compactness**: Many fewer features than image pixels
- **Locality**: Occupies small area; robust to clutter and occlusion

Harris Corner Detector

- Based on **image gradient** analysis using the **auto-correlation matrix M**
- For a corner: both eigenvalues (λ_1, λ_2) of M are large
- **Corner Response Functions**:
 - $R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$ ($k \approx 0.04-0.15$)

- $Q = (\lambda_1 \lambda_2) / (\lambda_1 + \lambda_2)$

Algorithm:

1. Compute M matrix for each image window
2. Find points with large corner response (R or $Q > \text{threshold}$)
3. Apply non-maximum suppression to keep local maxima

Limitation: Not invariant to scale changes

Scale-Invariant Interest Points

- Consider regions of different sizes around a point
- The characteristic scale is where a scale-space function achieves a maximum
- Use **Laplacian of Gaussian (LoG)** or its approximation **Difference of Gaussians (DoG)**

SIFT — Scale Invariant Feature Transform

A complete pipeline for scale- and rotation-invariant feature matching.

Overall Procedure:

1. **Scale-Space Extrema Detection:** Use DoG pyramid to find keypoints that are extrema in both space and scale
2. **Keypoint Localization:** Refine location and scale; eliminate edge-like keypoints
3. **Orientation Assignment:** Assign dominant gradient orientation to each keypoint
4. **Keypoint Description:** Build a 128-dimensional descriptor:
 - Extract a 16×16 window around the keypoint
 - Divide into 4×4 grid of cells (each 4×4 pixels)
 - Compute an 8-bin gradient orientation histogram per cell
 - Concatenate: $16 \text{ cells} \times 8 \text{ bins} = \mathbf{128\text{-element vector}}$

Properties: Invariant to translation, rotation, scale, and illumination changes

Matching: Compare descriptors using Euclidean distance; use threshold (e.g., 0.6) to filter matches

Other Descriptors: HOG (Histogram of Oriented Gradients), SURF, BRIEF

CV Lectures — Binary Image Analysis & Morphology

Binary Image Analysis Steps

1. **Thresholding:** Convert grayscale $\rightarrow$ binary
2. **Morphological Operators:** Clean up the binary image

3. **Connected Components:** Label distinct blobs
4. **Region Properties:** Describe each blob (area, perimeter, centroid, etc.)

Thresholding Methods

- **Simple Thresholding:** Fixed global threshold
- **Otsu's Method:** Automatically determines optimal threshold from the bimodal histogram (maximizes between-class variance)
- **Adaptive Thresholding:** Different thresholds for different image regions based on local neighborhoods
 - `ADAPTIVE_THRESH_MEAN_C`: threshold = local mean – C
 - `ADAPTIVE_THRESH_GAUSSIAN_C`: threshold = Gaussian-weighted local sum – C

Connected Components Analysis

- Labels each connected set of foreground pixels with a unique integer
- Uses the **row-by-row scan** algorithm:
 1. Scan pixels: if foreground and connected → propagate lowest label; if isolated → assign new label; track equivalences
 2. Relabel using equivalence table

Morphological Operators

Work on binary images using a **structuring element (SE)**:

- **Dilation:** Expands connected regions of 1s; fills holes and gaps; grows features
 - A pixel in the output is 1 if any pixel in the SE's neighborhood is 1 in the input
- **Erosion:** Shrinks connected regions; removes small protrusions and bridges
 - A pixel is 1 only if all pixels in the SE's neighborhood are 1 in the input
- **Opening** = Erosion followed by Dilation (same SE)
 - Removes small objects while preserving the shape of larger ones
 - **Idempotent:** repeated application has no further effect
- **Closing** = Dilation followed by Erosion (same SE)
 - Fills small holes while preserving the original shape
 - **Idempotent**

Basic Morphological Algorithms

- **Boundary Extraction:** $A - (A \ominus B)$ — subtract erosion from original
- **Region Filling:** Iteratively dilate within the complement of A
- **Connected Component Extraction:** via dilation from seed points
- **Thinning:** Progressively reduce regions to skeletal form

PRACTICAL / SHEETS SUMMARY

Sheet	Topic
-------	-------

Sheet 1	Histograms: construction, contrast, dynamic range; point operations (brightness reduction, intensity multiplication); Python thresholding without OpenCV built-ins
---------	--

Sheet 2	Applying filter kernels (smoothing, sharpening, edge detection) to images using OpenCV
---------	--

Sheet 3	Linear filter analysis: type identification, pixel value range, impulse response, correlation vs. convolution, isotropy; median vs. Gaussian for noise
---------	--

Sheet 4	Harris Corner Detector: computing the auto-correlation matrix M, Corner Response Function Q, applying threshold
---------	---

Sheet 5	SIFT descriptor structure (128-element vector from 4×4 grid of 8-bin histograms); distance-matrix-based matching with threshold; currency recognition using SIFT
---------	--

Sheet 6	Manual dilation/erosion computation; connected component analysis with equivalence table; morphological operations for object segmentation
---------	--

QUICK REFERENCE: KEY FORMULAS

Formula	Description
---------	-------------

$f(n) = g(n) + h(n)$	A* search evaluation function
----------------------	-------------------------------

$\hat{w} = (X^T X)^{-1} X^T y$	Linear regression closed-form solution
--------------------------------	--

$RSS(w) = (y - Xw)^T (y - Xw)$	Residual sum of squares
--------------------------------	-------------------------

$\text{Precision} = TP / (TP + FP)$	Classifier precision
-------------------------------------	----------------------

$\text{Recall} = TP / (TP + FN)$	Classifier recall / sensitivity
----------------------------------	---------------------------------

$F1 = 2PR / (P + R)$	Harmonic mean of precision and recall
----------------------	---------------------------------------

$\text{Gradient} = (\partial I / \partial x, \partial I / \partial y)$	Image gradient vector
--	-----------------------

$\text{Descriptor size} = 4 \times 4 \times 8 = 128$	SIFT descriptor dimension
--	---------------------------

$\text{Opening} = (A \ominus B) \oplus B$	Morphological opening
---	-----------------------

Formula	Description
Closing = $(A \oplus B) \ominus B$	Morphological closing
Boundary = $A - (A \ominus B)$	Boundary extraction